

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ

Escola Politécnica

Curso: Ciência da Computação

Disciplina: Métodos Quantitativos

Geração de Variáveis Aleatórias

Jafte Carneiro Fagundes da Silva

Atividade A16

01 de junho de 2026

Sumário

1	Introdução	2
2	Situações que Usam Geradores de Números Aleatórios	3
2.1	Simulação de Sistemas Complexos	3
2.2	Criptografia e Segurança	3
3	Método de Geração de Números Aleatórios: Linear Congruente	4
3.1	Descrição do Método	4
3.2	Exemplo Prático	4
3.3	Vantagens e Desvantagens	4
4	Avaliação da Qualidade de Sequências de Números Aleatórios	6
4.1	Testes Estatísticos Básicos	6
4.1.1	Teste de Uniformidade (Teste de Kolmogorov-Smirnov)	6
4.1.2	Teste do Qui-Quadrado	6
4.1.3	Teste de Autocorrelação	6
4.2	Testes Estatísticos Avançados	6
4.2.1	Teste de Frequência (Monobit Test)	6
4.2.2	Teste de Runs	7
4.2.3	Teste de Entropia	7
4.3	Suites de Teste Conhecidas	7
5	Mersenne Twister	8
5.1	História e Características	8
5.2	Algoritmo	8
5.2.1	1. Inicialização (Seeding)	8
5.2.2	2. Geração de Números	8
5.3	Implementação Conceitual	9
5.4	Vantagens	10
5.5	Limitações	10
5.6	Variantes	10
6	Conclusão	11

1 Introdução

Variáveis aleatórias são conceitos fundamentais em probabilidade e estatística. A geração de números aleatórios é essencial em diversas aplicações computacionais, desde simulações científicas até criptografia. Este trabalho apresenta uma pesquisa abrangente sobre técnicas de geração de variáveis aleatórias, métodos para sua avaliação e o funcionamento de geradores modernos.

2 Situações que Usam Geradores de Números Aleatórios

2.1 Simulação de Sistemas Complexos

A simulação de Monte Carlo é uma das aplicações mais importantes de geradores de números aleatórios. Utilizando sequências aleatórias, é possível simular comportamentos de sistemas físicos, biológicos e econômicos que seriam difíceis ou impossíveis de resolver analiticamente. Exemplos incluem:

- Simulação do comportamento de partículas em física quântica;
- Avaliação de risco em portfólios de investimento;
- Modelagem de fenômenos naturais como movimentos de fluidos ou propagação de radiação;
- Estimativa de integrais definidas através da amostragem aleatória.

Na simulação de Monte Carlo, gera-se uma grande quantidade de números aleatórios distribuídos de acordo com probabilidades específicas do sistema. Com suficientes amostras, a distribuição dos resultados converge para valores reais do sistema, permitindo estimativas precisas de grandezas difíceis de calcular diretamente.

2.2 Criptografia e Segurança

Geradores de números aleatórios são fundamentais em criptografia moderna. Números verdadeiramente aleatórios (ou pseudo-aleatórios de alta qualidade) são necessários para:

- Geração de chaves criptográficas seguras em algoritmos de encriptação como RSA e AES;
- Criação de vetores de inicialização (IV) em modos de cifra por bloco;
- Geração de nonces e tokens de autenticação únicos;
- Salting em funções de hash criptográficas para proteção de senhas.

A qualidade do gerador de números aleatórios em aplicações criptográficas é crítica. Um gerador fraco ou previsível comprometeria toda a segurança do sistema. Por isso, aplicações de segurança utilizam geradores criptograficamente seguros, que passam por rigorosos testes estatísticos.

3 Método de Geração de Números Aleatórios: Linear Congruente

3.1 Descrição do Método

O Gerador Linear Congruente (LCG) é um dos métodos mais antigos e simples para gerar números pseudo-aleatórios. Sua fórmula é:

$$X_{n+1} = (a \cdot X_n + c) \mod m \quad (1)$$

Onde:

- X_n é o n -ésimo número gerado;
- a é o multiplicador;
- c é o incremento;
- m é o módulo;
- X_0 é a semente inicial.

Para gerar números no intervalo $[0, 1)$, normaliza-se o resultado:

$$U_n = \frac{X_n}{m} \quad (2)$$

3.2 Exemplo Prático

Considere os parâmetros: $a = 1103515245$, $c = 12345$, $m = 2^{31}$ e $X_0 = 1$. A sequência gerada seria:

$$X_1 = (1103515245 \times 1 + 12345) \mod 2^{31} = 1103527590$$

$$U_1 = \frac{1103527590}{2^{31}} \approx 0.5140$$

$$X_2 = (1103515245 \times 1103527590 + 12345) \mod 2^{31} = 377401575$$

$$U_2 = \frac{377401575}{2^{31}} \approx 0.1757$$

3.3 Vantagens e Desvantagens

Vantagens:

- Simples de implementar;

- Rápido computacionalmente;
- Completamente determinístico e reproduzível.

Desvantagens:

- Qualidade estatística inferior comparado a métodos modernos;
- Períodos curtos com certos parâmetros;
- Números sucessivos apresentam correlação (especialmente os bits menos significativos);
- Não adequado para aplicações criptográficas;
- Padrões de baixa dimensionalidade (problemas com pares e tríadas consecutivas).

4 Avaliação da Qualidade de Sequências de Números Aleatórios

4.1 Testes Estatísticos Básicos

Para avaliar se uma sequência de números aleatórios é adequada para aplicações, diversos testes estatísticos podem ser aplicados:

4.1.1 Teste de Uniformidade (Teste de Kolmogorov-Smirnov)

Verifica se a sequência segue uma distribuição uniforme no intervalo $[0, 1)$. A estatística do teste é:

$$D = \max |F_n(x) - F(x)| \quad (3)$$

Onde $F_n(x)$ é a função de distribuição empírica e $F(x)$ é a função de distribuição teórica. Um valor pequeno de D indica boa uniformidade.

4.1.2 Teste do Qui-Quadrado

Divide o intervalo $[0, 1)$ em k subintervalos iguais, conta quantos números caem em cada intervalo, e compara com a frequência esperada:

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i} \quad (4)$$

Onde O_i é a frequência observada e E_i é a frequência esperada. A sequência é considerada adequada se χ^2 não exceder o valor crítico da tabela chi-quadrado.

4.1.3 Teste de Autocorrelação

Verifica se há dependência entre números consecutivos ou números separados por um lag τ . A autocorrelação é:

$$\rho_\tau = \frac{\sum_{i=1}^{n-\tau} (U_i - \bar{U})(U_{i+\tau} - \bar{U})}{\sum_{i=1}^n (U_i - \bar{U})^2} \quad (5)$$

Uma sequência de qualidade deve ter ρ_τ próximo de zero.

4.2 Testes Estatísticos Avançados

4.2.1 Teste de Frequência (Monobit Test)

Verifica se existem quantidades aproximadamente iguais de zeros e uns na representação binária dos números aleatórios.

4.2.2 Teste de Runs

Avalia se há mudanças apropriadas entre valores consecutivos na sequência. Runs muito longos indicam falta de aleatoriedade.

4.2.3 Teste de Entropia

Calcula a entropia de Shannon da sequência. Uma sequência verdadeiramente aleatória deve ter entropia máxima para seu conjunto de símbolos.

4.3 Suites de Teste Conhecidas

Existem suites de testes padronizadas:

- **Diehard Tests:** 15 testes estatísticos para geradores de números aleatórios;
- **NIST Statistical Test Suite:** Suite desenvolvida pelo NIST com 15 testes para avaliação criptográfica;
- **TestU01:** Biblioteca em C com centenas de testes incluindo os anteriores.

5 Mersenne Twister

5.1 História e Características

O Mersenne Twister é um gerador de números pseudo-aleatórios desenvolvido por Matsumoto e Nishimura em 1997. É amplamente utilizado em aplicações científicas e em muitas linguagens de programação (Python, Ruby, C++11, etc.). O nome se deve ao fato de usar números primos de Mersenne.

Características principais:

- Período muito longo: $2^{19937} - 1$ (aproximadamente 10^{6000});
- Excelentes propriedades estatísticas de 623 dimensões;
- Rápido: gera 32 bits (ou 64 bits na versão MT19937-64) por iteração;
- Não é adequado para criptografia, pois é previsível com poucos valores observados;
- Implementação simples e bem documentada.

5.2 Algoritmo

O Mersenne Twister funciona sobre um vetor de estado de 624 valores de 32 bits. O algoritmo pode ser resumido em:

5.2.1 1. Inicialização (Seeding)

Dado uma semente x_0 , inicializa-se o vetor de estado $MT[0..623]$:

$$MT[i] = f(i, MT[i - 1]) \quad (6)$$

Onde f é uma função de inicialização que garante boa distribuição do estado inicial.

5.2.2 2. Geração de Números

O processo ocorre em duas fases:

Fase 1: Twist (Recorrência Linear)

A cada iteração, atualiza-se o vetor de estado usando uma operação de recorrência baseada em números primos de Mersenne:

$$MT[i] = MT[(i + m) \bmod n] \oplus ((\text{upper}(MT[i]) \mid \text{lower}(MT[i + 1])) \gg 1) \oplus A \quad (7)$$

Onde:

- $n = 624$ é o tamanho do vetor de estado;
- $m = 397$ é um parâmetro de deslocamento;
- $\oplus$ é a operação XOR;
- $\gg$ é shift (deslocamento) à direita;
- A é uma matriz constante;
- `upper` e `lower` extraem os bits mais e menos significativos.

Fase 2: Tempering (Temperagem)

O valor extraído sofre transformações (tempering) para melhorar propriedades estatísticas:

$$y = \text{MT}[i] \quad (8)$$

$$y = y \oplus (y \gg 11) \quad (9)$$

$$y = y \oplus ((y \ll 7) \text{ AND } 0x9d2c5680) \quad (10)$$

$$y = y \oplus ((y \ll 15) \text{ AND } 0xefc60000) \quad (11)$$

$$y = y \oplus (y \gg 18) \quad (12)$$

O valor y é então retornado como o número pseudo-aleatório.

5.3 Implementação Conceitual

O pseudocódigo básico é:

```
função inicializa(semente):
    MT[0] = semente
    para i de 1 até 623:
        MT[i] = lowestW bits de (f * (MT[i-1] ^ (MT[i-1] >> (W-2))) + i

função extrai_número():
    se índice >= 624:
        twist()
    y = MT[índice]
    y = tempering(y)
    índice = índice + 1
    retorna y

função twist():
```

```

para i de 0 até 623:
    x = (MT[i] & UPPER_MASK) | (MT[(i+1) mod 624] & LOWER_MASK)
    MT[i] = MT[(i+397) mod 624] ^ (x >> 1) ^ ((x & 1) * 0x9908b0df)
índice = 0

```

5.4 Vantagens

- Período extremamente longo ($2^{19937} - 1$);
- Distribuição uniforme em 623 dimensões;
- Rápido e eficiente;
- Implementação simples e disponível em muitas linguagens;
- Estável e amplamente testado;
- Produz sequências de alta qualidade para simulações científicas.

5.5 Limitações

- Não é criptograficamente seguro (pode ser predito com observação de 624 valores consecutivos);
- Usa muita memória (624×4 bytes = 2.5 KB apenas para estado);
- Inicialização simples pode produzir correlações em múltiplas instâncias com sementes próximas;
- Alguns bits têm períodos menores que o período total.

5.6 Variantes

Existem variantes do Mersenne Twister para diferentes propósitos:

- **MT19937-64**: Versão de 64 bits com período $2^{19937} - 1$;
- **SFMT (SIMD-oriented Fast Mersenne Twister)**: Versão otimizada para processadores modernos com instruções SIMD;
- **dSFMT**: Variante que gera números em ponto flutuante diretamente.

6 Conclusão

A geração de números aleatórios é uma área fundamental da computação com aplicações vastas em simulação, criptografia e análise estatística. Desde métodos simples como o gerador linear congruente até algoritmos sofisticados como o Mersenne Twister, a evolução desses geradores reflete a importância crescente da qualidade e eficiência em aplicações modernas.

O Mersenne Twister se consolidou como um padrão de fato para aplicações científicas devido ao seu excelente balance entre período longo, qualidade estatística e velocidade de execução. Entretanto, compreender limitações e aplicar testes estatísticos apropriados permanece essencial para garantir que a sequência gerada seja adequada para cada contexto específico.

Referências

- [1] Wikipedia. **Variável Aleatória**. Disponível em: https://pt.wikipedia.org/wiki/Varivel_aleatria. Acesso em: 1 jun. 2026.
- [2] Wikipedia. **Mersenne Twister**. Disponível em: https://en.wikipedia.org/wiki/Mersenne_Twister. Acesso em: 1 jun. 2026.
- [3] Matsumoto, M.; Nishimura, T. **Mersenne Twister: a 623-dimensionally equidistributed uniform pseudo-random number generator**. *ACM Transactions on Modeling and Computer Simulation*, v. 8, n. 1, p. 3-30, 1998.
- [4] Saito, M.; Matsumoto, M. **SIMD-oriented Fast Mersenne Twister: a 128-bit Pseudo-random Number Generator**. In: *International Conference on Monte Carlo Methods and Applications*. 2008.
- [5] NIST. **A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications**. Special Publication 800-22 Rev. 3a. Disponível em: <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-22r1a.pdf>. 2010.
- [6] Knuth, D. E. **The art of computer programming: Volume 2, Seminumerical algorithms**. 3. ed. Addison-Wesley, 1997.
- [7] L'Ecuyer, P. **Handbook of Computational Statistics: Concepts and Methods**. In: *Pseudo-Random Number Generators*. Springer, 2006. Cap. 3.
- [8] NVIDIA. **Mersenne Twister Applied to Surface Scattering**. CUDA SDK. 2010.